

# Web Application Workflow

---

Ornella Irrera

Department of Information Engineering, University of Padova

May 31, 2024

# Outline

---

- The main steps to follow to develop a web application from scratch
  - The data layer
  - The Business layer
  - The presentation layer
- An example of web application

# The three tiers architecture

---

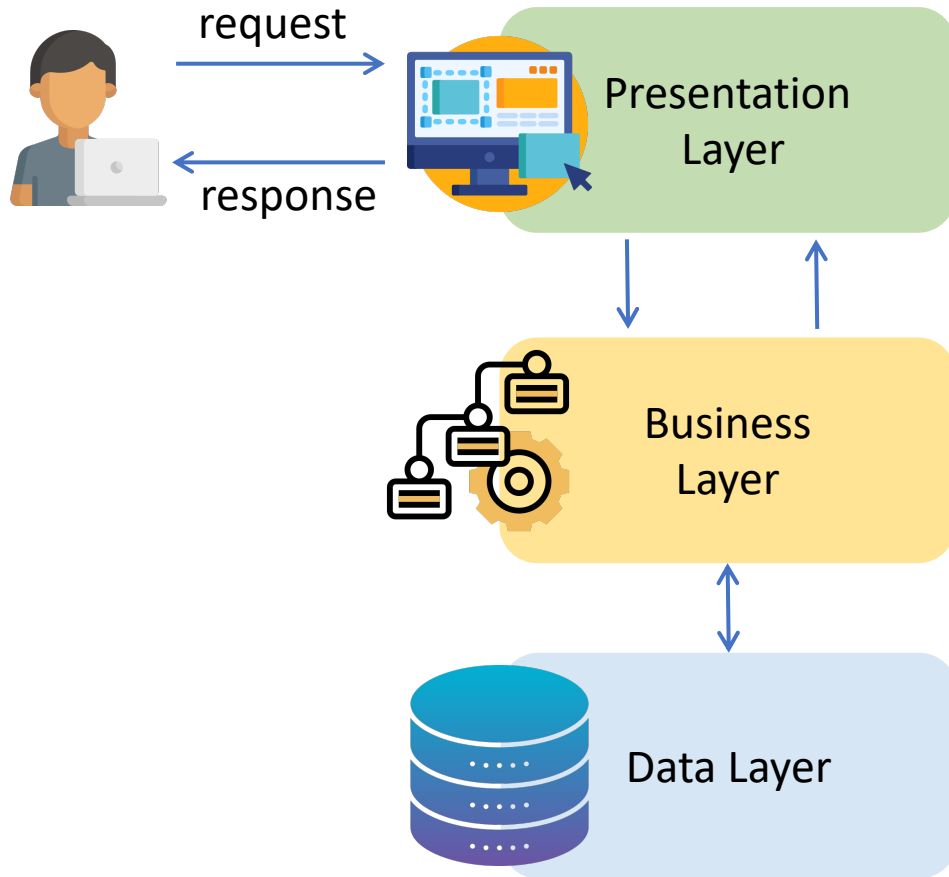
**Web applications** are instances of three tiers architectures.

The three-tier architecture is a client-server design pattern that divides an application into three distinct logical and physical layers: the **presentation layer**, the **business layer**, and the **data layer**.

- **Data layer:** The data tier is responsible for storing, retrieving, and managing the data.  
*Technology:* PostgreSQL databases
- **Business layer:** The business layer defines and **control the flow of operations** and defines operations on data and constraints;  
*Technology:* APIs, servlets, jsps, web servers
- **Presentation layer:** Ensures the persistency of data, searches and retrieves the data;  
*Technology:* HTML, CSS, JS

# The three tiers architecture

---



The user interacts through the presentation layer

User actions trigger requests that the presentation tier sends to the logic tier.

The logic layer processes these requests, applies business logic, and interacts with the data tier to fetch or store data.

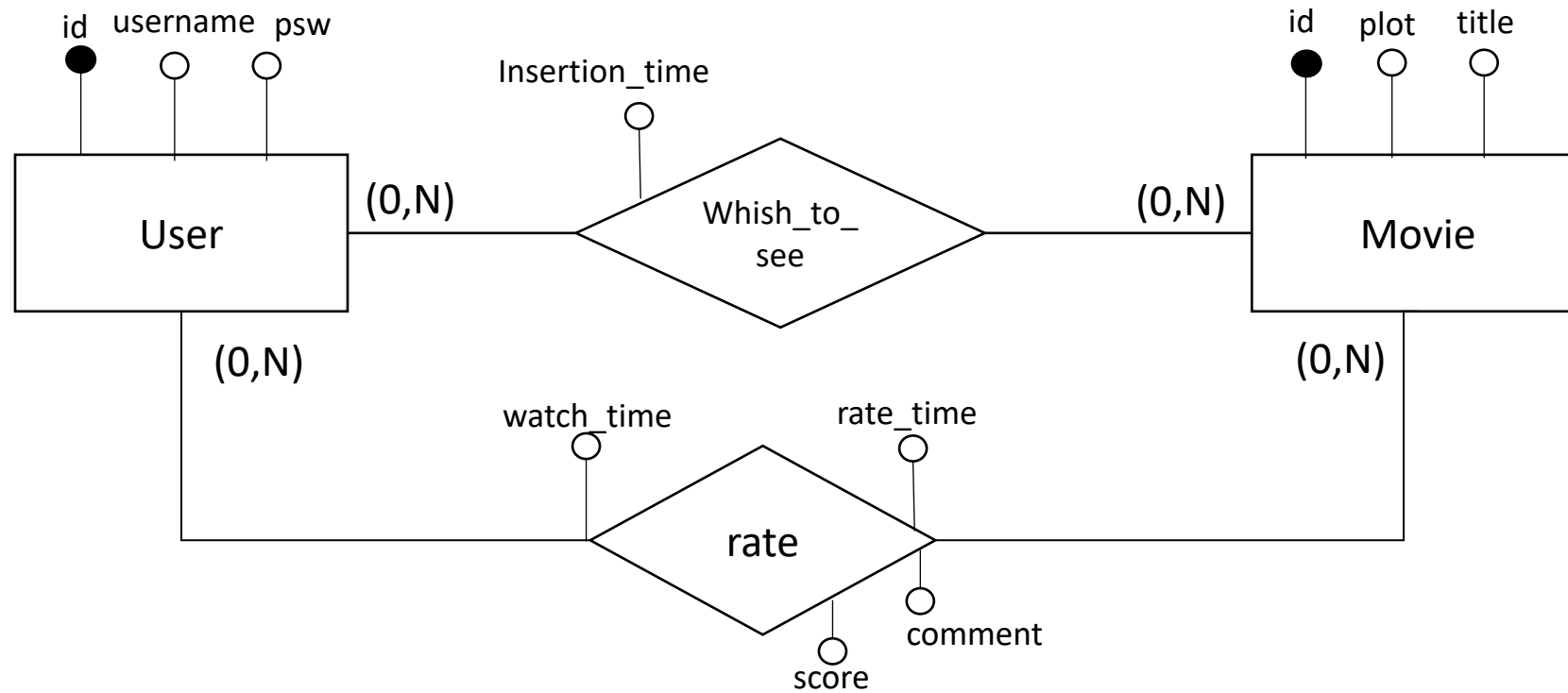
The data layer sends the required data back to the business layer, which processes it and sends the appropriate response to the presentation layer.

# The Data Logic

---

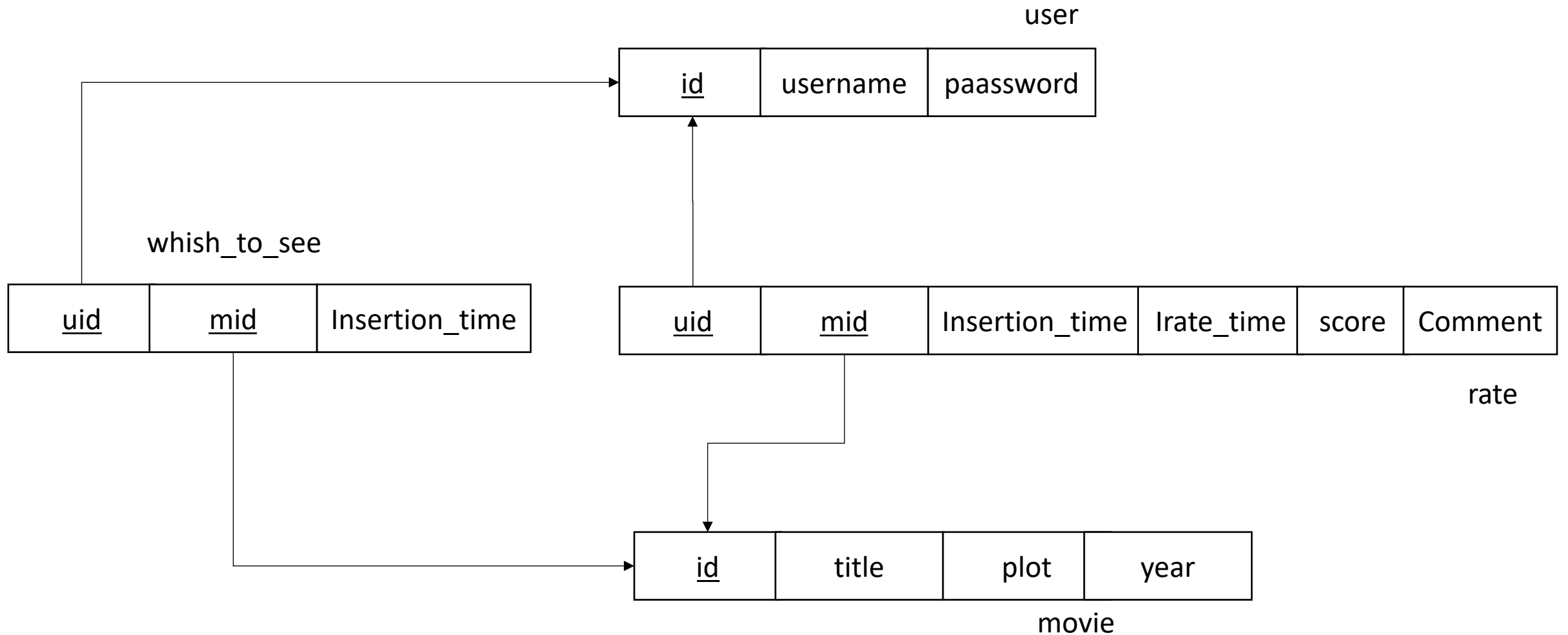
# ER Schema and Relational Schema

---



# ER Schema and Relational Schema

---



# The Business Logic

---



# Interacting with the DB

---

I have correctly instantiated my database: I have created and populated my tables.

I am ready to connect to it and start retrieving and handling data.

How does this work in a web application?

At this point we have to:

- Define our resources;
- Implement our DAO;
- Setup the connection;

# Interacting with the DB – the **resources**

---

A **resource** is a java class that is used to represent various kinds of objects or entities in a web application.

It can represent anything that is described by a specific set of properties that can describe its **state**.

Example of resources are:

- The **message class** used to represent messages sent to the user
- The **user class** used to represent the user of the web application (characterized by a name, surname ...)

```
public class User {
```

2 usages

```
private final Long id;
```

2 usages

```
private final String username;
```

2 usages

```
private final String password;
```

Ornellalrrera

```
public User(Long id, String username, String password) {
    this.id = id;
    this.username = username;
    this.password = password;
}
```

Ornellalrrera

```
public final String getUsername() { return username; }
```

Ornellalrrera

```
public final Long getId() { return id; }
```

1 usage Ornellalrrera

```
public final String getPassword() { return password; }
```

}

Set of properties used to describe the user.

Constructor. Sometimes there is more than one constructor. In the Message class for example we distinguish between informative and error messages.

"getters" methods to access to the private properties Of the User resource

# Interacting with the DB – the DAO

---

Data Access Objects (DAOs) are a design pattern used in web applications to abstract and encapsulate all access to the data source.

Thanks to DAOs, the business logic can interact with the data through method calls, **without knowing the underlying data access details**, creating a higher level of abstraction. This abstraction allows for flexibility in changing the **data source** or data access technology without impacting the business logic.

→ SEPARATION OF CONCERNS

# the DAO in practice

---

```
1 usage  OrnellaIrrera *
public class MovieInfoDAO extends AbstractDAO<Movie> {
    1 usage
    private static final String STATEMENT_MOVIE_INFO = "SELECT id,title,plot,year FROM public.movie where id = ?";
    2 usages
    private final Long mid;
    1 usage  OrnellaIrrera
    public MovieInfoDAO(final Connection con, final Long mid) {
        super(con);
        this.mid = mid;
    }
}
```

Each DAO extends the **AbstractDAOClass**.

The AbstractDAOClass is the one that provides the uniform behaviour shared by all the DAOs extending it.

The AbstractDAOClass takes care of closing the connection and rolling back if needed.

# the DAO in practice

---

```
2 usages  | OrnellaIrrera *
@Override
protected void doAccess() throws Exception {

    PreparedStatement stmtnt = null;
    ResultSet rs = null;
    Movie movie = null;
    try {
        stmtnt = con.prepareStatement(STATEMENT_MOVIE_INFO);
        stmtnt.setLong( parameterIndex: 1, mid);
        rs = stmtnt.executeQuery();
        if(rs.next()){
            movie = new Movie(rs.getLong( columnLabel: "id"),rs.getString( columnLabel: "title"),rs.getString( columnLabel: "plot"),
        }
    } finally {
        if (rs != null) {
            rs.close();
        }
        if (stmtnt != null) {
            stmtnt.close();
        }
    }
    this.outputParam = movie;
}
```

Each DAO implements the **doAccess()** method which implements the actual logic to perform.

In this case the **doAccess()** method executes the statements to retrieve the information of a movie, creates a new **Movie** (our **resource** previously defined) resource and return it as output.

# The context.xml file

---

In the context.xml file we configure the access to the database.

```
<Resource name="jdbc/movie_rating"
  auth="Container"
  type="javax.sql.DataSource"
  factory="org.apache.tomcat.jdbc.pool.DataSourceFactory"
  driverClassName="org.postgresql.Driver"
  url="jdbc:postgresql://localhost:5432/movie_db"
  username="ornella"
  password="ornella"

  testOnBorrow="true"
  validationQuery="SELECT 1"
  timeBetweenEvictionRunsMillis="30000"
  maxActive="2"
  minIdle="1"
  maxWait="10000"
  initialSize="1"
  removeAbandonedTimeout="60"
  removeAbandoned="true"
  closeMethod="close"
/>
```

This file will be used by the web server where your web application is running to instantiate the connection to the database.

\*The resource name must be also reported in the AbstractDatabaseServlet in the data source definition.

# Java Servlets

---

Once that we defined our resources, the operations to perform on the Database, we have to create our **servlets**.

A **servlet** is a java-based web component run by a **web-server** (i.e., tomcat).

Servlets take care of:

- Handling requests from web browsers;
- Processing the data sent by the user;
- Interacting with the database to retrieve/update data;
- Update resources' states (if needed);
- Sending back responses.



# The web.xml file

---

Each servlet is mapped to the corresponding **URI**.

This mapping is defined in the web.xml file: a configuration file where the servlets are mapped to the corresponding URI.

```
<!-- Servlet mapping -->
<servlet>
    <servlet-name>MovieServlet</servlet-name>
    <servlet-class>it.unipd.dei.webapp.movieratings.servlet.MovieServlet</servlet-class>
</servlet>
<servlet-mapping>
    <servlet-name>MovieServlet</servlet-name>
    <url-pattern>/movies/*</url-pattern>
</servlet-mapping>
```

# Java Servlets – parameters and attributes

---

`doGet(request,response)` → handles HTTP GET requests from the browser

`doPost(request,response)` → handles HTTP POST requests from the browser

**PARAMETERS:** parameters refer to pieces of data that are sent along with an HTTP request. These parameters can be included in the URL itself (query parameters) or sent in the body of the request (e.g., form parameters in a POST request).

```
String username = request.getParameter("username");
```

**ATTRIBUTES:** they are objects that servlets use to store and share information across different parts of a web application. These attributes can be useful for passing data between servlets and jsp.

```
request.setAttribute("username","Ornella");
```

# Java Servlets – Request handling

---

Three main techniques used in request management in web applications are **forwarding**, **inclusion**, and **redirection**.

- **Forward** → the request is passed from a servlet to another (or to a jsp page) without notifying the client. We stay on the server-side.
- **Include** → the content of another servlet/jsp is included during the processing of the request, enabling code reuse. We stay on the server-side.
- **Redirection** → client receives this response and makes a new request to the specified URL, changing the URL in the browser's address bar. We move between server and client.

# Java Servlets

```
@WebServlet(name = "MovieServlet", value = "/movies/info/*")
public class MovieServlet extends AbstractDatabaseServlet {
    no usages  OrnellaIrrera *
    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException {
        // get both student and group info
        LogContext.setIPAddress(request.getRemoteAddr());
        HttpSession session = request.getSession();
        String op = request.getParameter(s: "movie");

        try {
            if (session.getAttribute(s: "user") == null){
                Message m = new Message(
                    message: "User not login",
                    errorCode: "E400", errorDetails: "User not found");
                ErrorCode ec = ErrorCode.INTERNAL_ERROR;
                LOGGER.error("An error occurred, no user logged in");

                response.setStatus(ec.getHTTPCode());
                request.setAttribute(s: "message", m);
                request.getRequestDispatcher(s: "/jsp/login.jsp").forward(request, response);
            }
        }
    }
}
```

The movie id is a parameter of the request.

`http://.../info?movie=5`

This action “transfers” the control from a servlet to another or to another resource (i.e., to a jsp page)

# JSP

---

The user of the web application usually interacts with HTML, CSS and JS that allow him to trigger event and issue request.

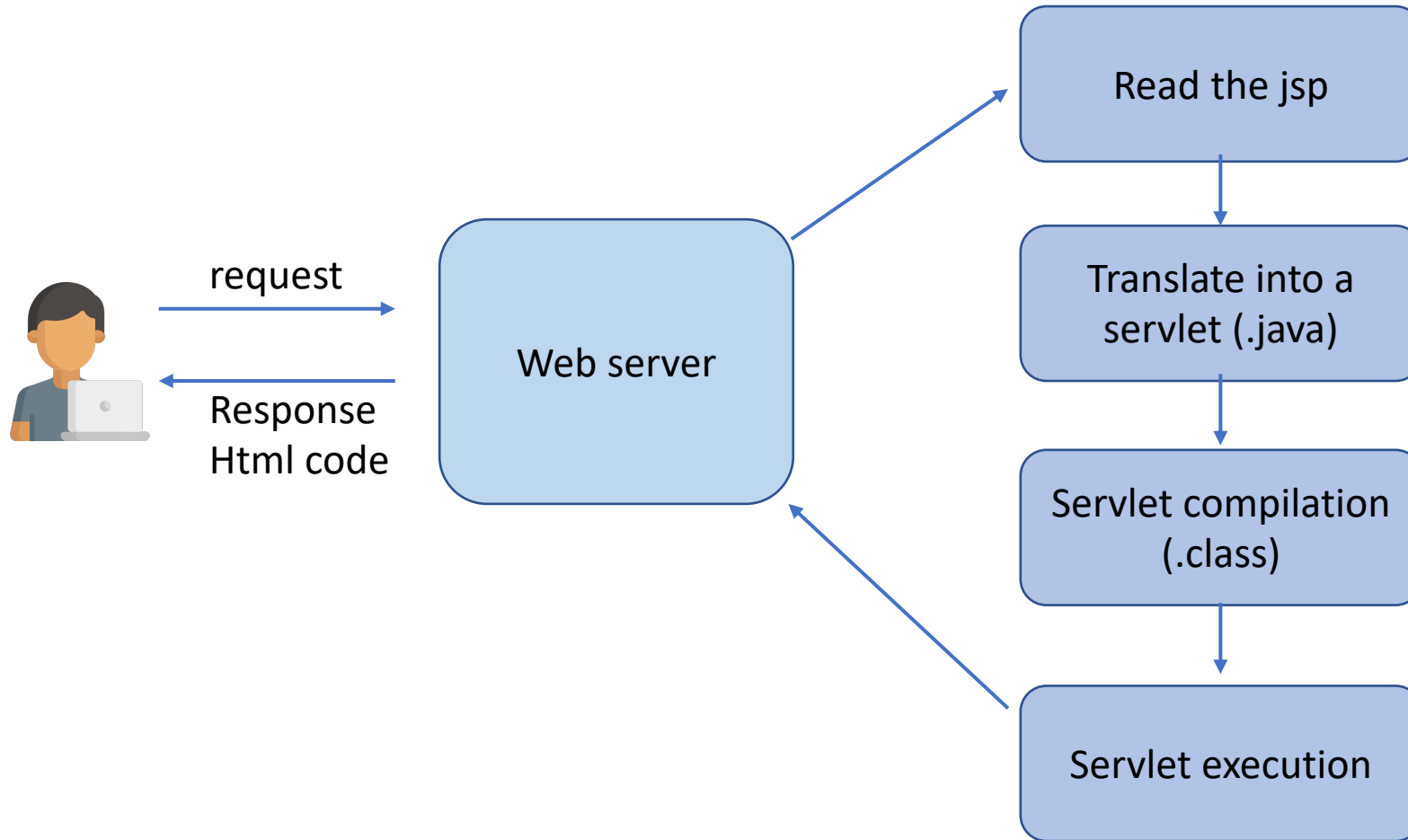
The generation of HTML code directly from the servlet is not the best option as it affect ease of maintenance and readability.

In this context **JSPs** play a fundamental role as they are serve-side technologies that enables the generation of dynamic content.

JSPs **rely on Java Beans** (i.e., Message, User, Movie) to exchange information between the web app components (e.g., the jsp and the servlet)

# JSP – Lifecycle

---



# JSP – an example

```
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<%@ page contentType="text/html" pageEncoding="UTF-8" %>

<!DOCTYPE html>
<html>
<head>
  <%@ include file="/html/cdn.html"%>
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <meta charset="ISO-8859-1">
  <title>Login</title>
  <script src="${pageContext.request.contextPath}/js/login.js"></script>
```

Directives providing global information

**Static** inclusion → the resource is included at translation time

**Dynamic** inclusion <jsp:include> → the inclusion happens at runtime

When we incorporate script or link to style sheet, relying on the relative path is not effective: `${pageContext.request.contextPath}` is used to obtain the context path of the current request. For example, if your web application's context is `http://localhost:8080/myapp`, then `${pageContext.request.contextPath}` would return `/myapp`.

```
<c:url value="/login"/></c:url>
```

With `c:url` we can obtain the similar result

```
</nav>
<h1>Welcome back <c:out value="${sessionScope.user.getUsername()}" /></h1>
<hr/>
```

Access to session data

# JSP – an example

---

```
    <input type="password" name="password" class="form-control" placeholder="Password">
  </div>
  <button type="submit" class="btn btn-black login">Login</button>
</form>
<br/>
<c:choose>
  <c:when test="${message.error}">
    <div class="msgmargin">
      <div class="alert alert-danger" role="alert">
        <span>
          <c:out value="${message.message}"/>
        </span>
      </div>
    </div>
  </c:when>
  <c:otherwise></c:otherwise>
</c:choose>
```

With `${}` we access to specific variable values in the request scope or java beans properties



# The Model-View Controller paradigm

---

The MVC paradigm separates the concerns of an application into three interconnected components.

3 main components:

- The **model**: it is the internal representation of information – typically the **resources (or java beans)**.
- The **view**: the web interface that presents the information the users interact with.
- The **controller**: the java servlets: it receives user input from the **view**, processes it also interacting with the **model**, and returns the output to the **view**.

Example:

The user wants to check the list of movies he has in the list of movies to watch in the future.

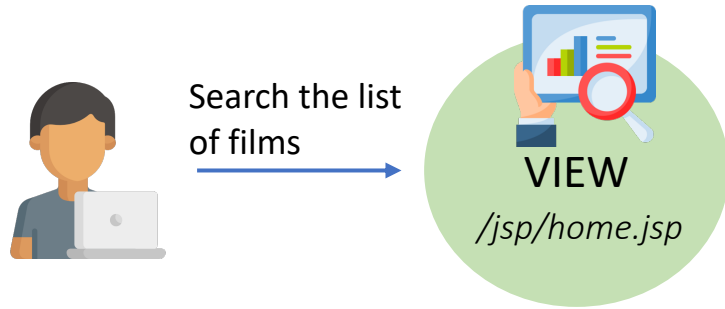
My **model** is: the user and the movie;

My **controller** is my servlet which process the request of the user and generates new output lists;

My **view** is the jsp page of the home of a user.

# The Model-View Controller paradigm

---



The user interacts with the jsp and triggers the retrieval of his list of movies

# The Model-View Controller paradigm

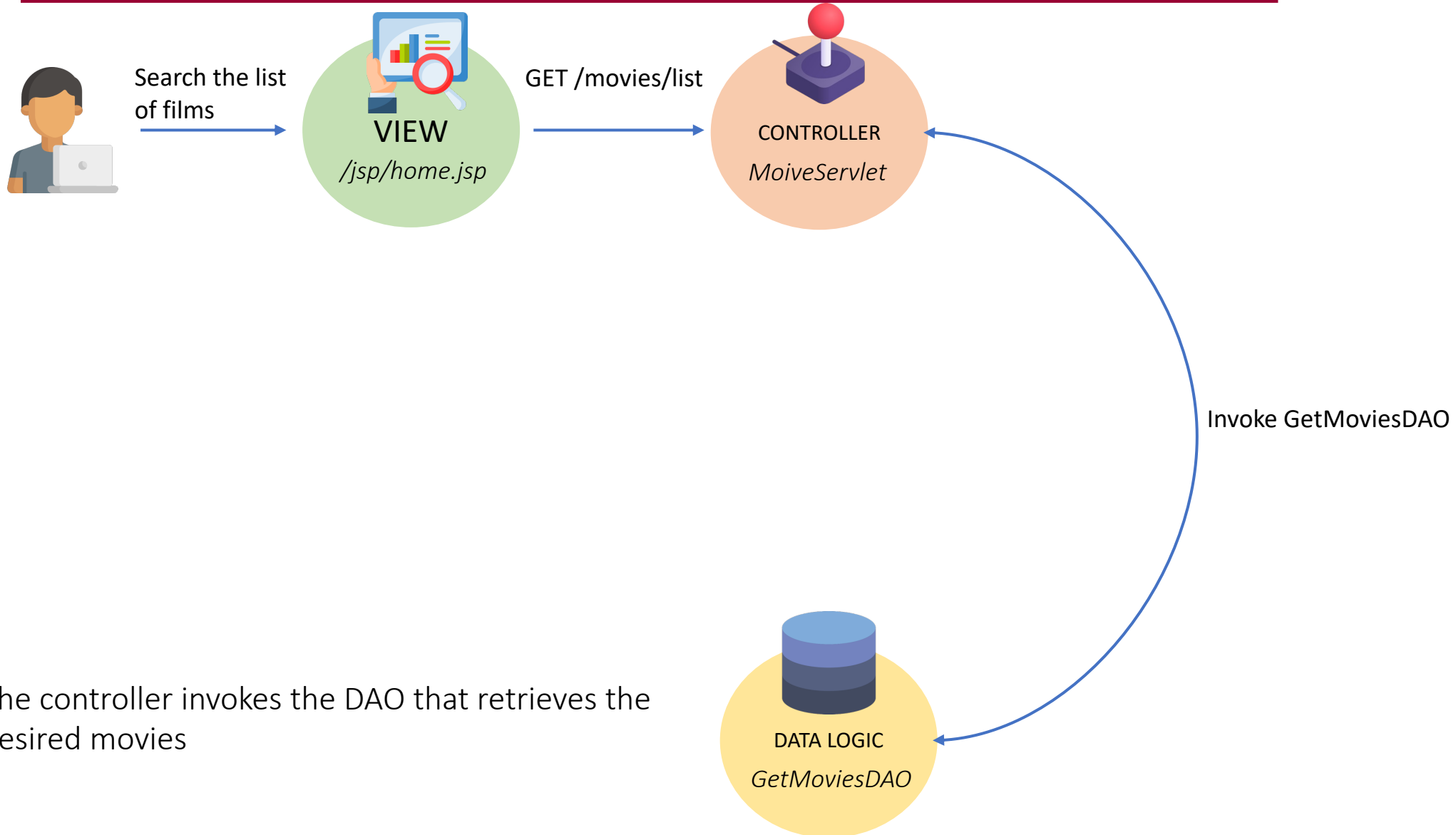
---



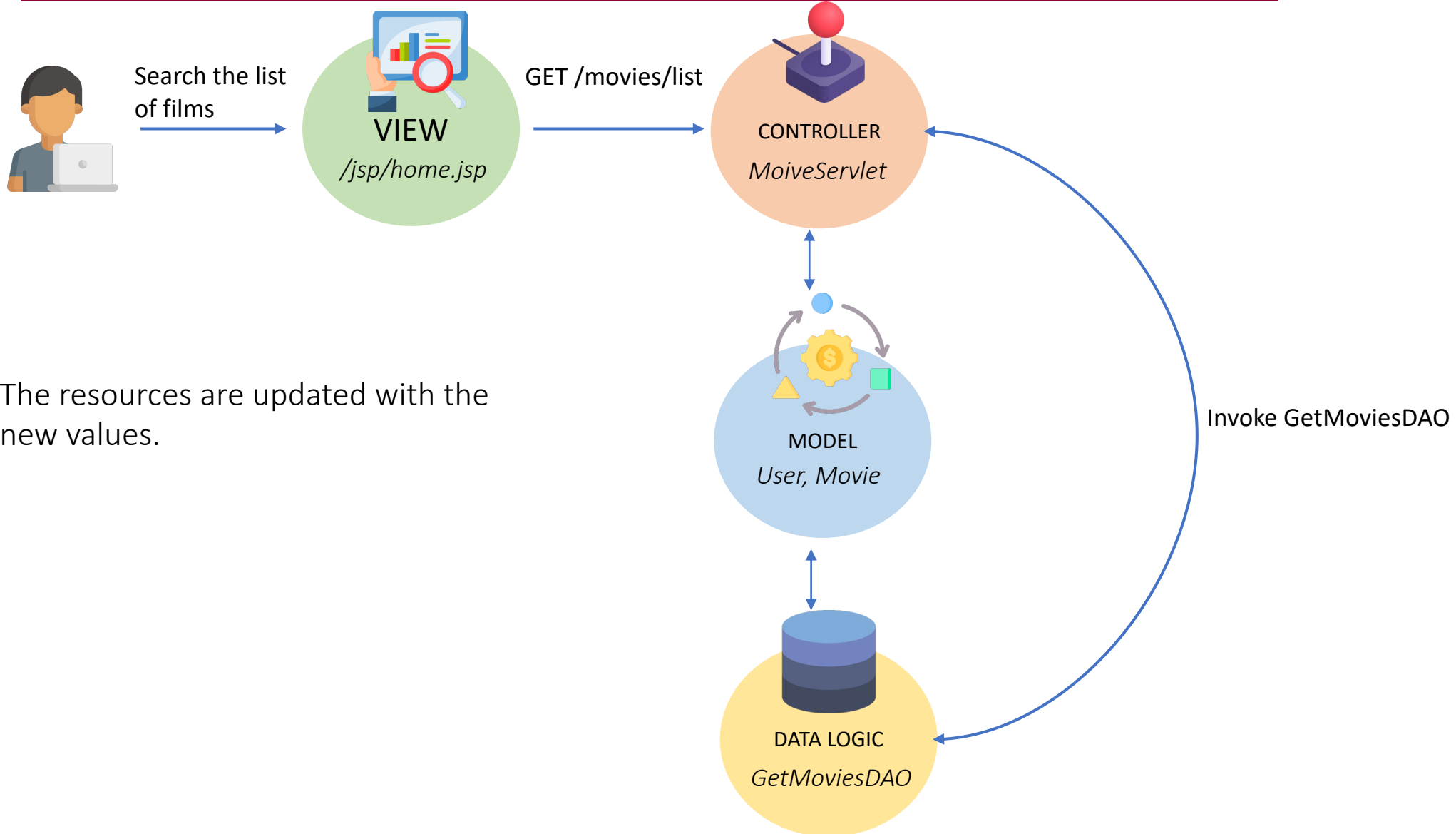
The **controller**, the `MovieServlet`, receives the GET request and process it.

# The Model-View Controller paradigm

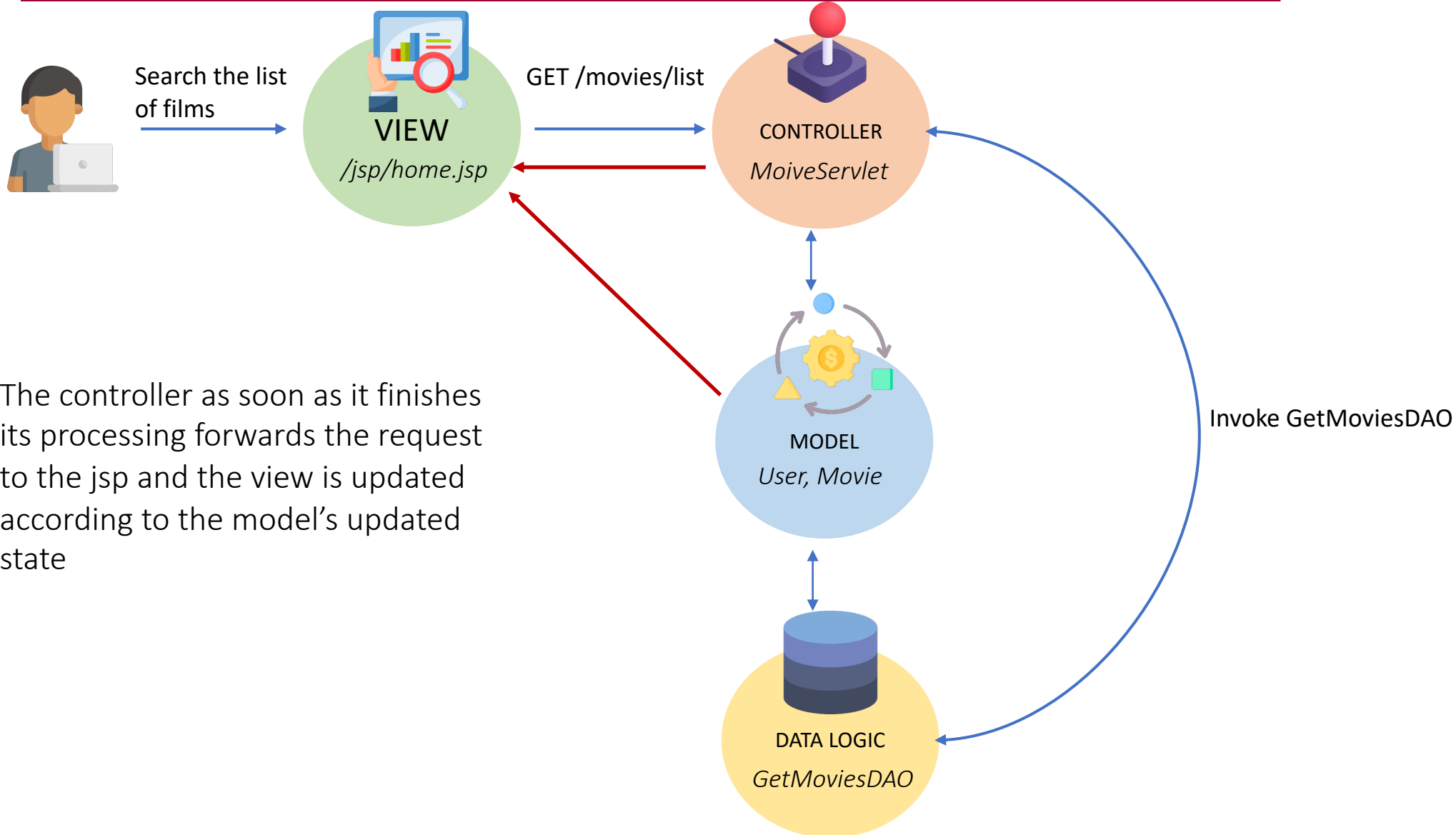
---



# The Model-View Controller paradigm



# The Model-View Controller paradigm



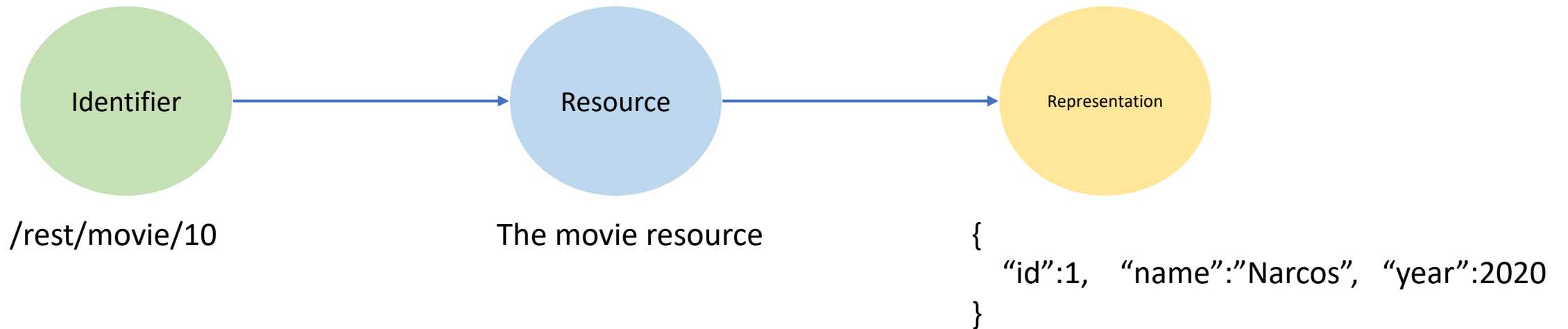
The controller as soon as it finishes its processing forwards the request to the jsp and the view is updated according to the model's updated state

# REST

---

REST is an architectural paradigm which applies the architectural principles of the web to the web services.

An important concept of REST is the **resource**



REST is stateless: all the information to satisfy the request must be in the URI.

# REST – methods and HTTP

---

REST is designed to communicate over HTTP protocol

In order to properly manage a resource, and:

- Create a new resource
  - Read a resource
  - Update a resource
  - Delete a resource
- 
- CRUD OPERATIONS**

REST relies on standard HTTP methods and performs operations on resources:

- CREATE → HTTP POST
- READ → HTTP GET
- UPDATE → HTTP PUT
- DELETE → HTTP DELETE



# REST

---

```
public SearchReviewsByMovieIDRR(final HttpServletRequest req, final HttpServletResponse res, Connection con) {
    super(Actions.GET_REVIEWS_BY_ID, req, res, con);
}

1 usage  • OrnellaIrrera •
@Override
protected void doServe() throws IOException {

    ArrayList<Review> reviews;
    Message m;

    try {
        String path = req.getRequestURI();
        path = path.substring( beginIndex: path.lastIndexOf( str: "reviews") + 8);

        final Long mid = Long.parseLong(path);

        LogContext.setResource(path);
        reviews = new GetMovieRatingsDAO(con,mid).access().getOutputParam();
        LOGGER.info( message: "movie",mid);

        if (reviews != null) {
            LOGGER.info( message: "Reviews %d successfully retrieved",mid);

            res.setStatus(HttpServletResponse.SC_OK);
            new ResourceList(reviews).toJSON(res.getOutputStream());
        } else { // it should not happen
            LOGGER.error( message: "Fatal error while searching review(s) %d.",mid);

            m = new Message( message: "Movie not found.", errorCode: "E5A1", errorDetails: null);
            res.setStatus(HttpServletResponse.SC_INTERNAL_SERVER_ERROR);
            m.toJSON(res.getOutputStream());
        }
    }
}
```

Resource URI: /rest/reviews/{id}

Get the reviews related to that specific id.

We return the json representation of the list of the reviews

# How can we debug a web application?

---

When we develop a web application it is important to keep track of what users do to:

- Keep track of bad behaviors
- Keep track of error conditions, being able to correctly identify the cause.

We are not able to debug line by line.

Tomcat, our webserver, has a file: **logs/catalina.out** where the logs of all the web apps deployed in your tomcat are stored.

These logs are very general, and so it is difficult to keep track of all the possible interactions.

# The catalina.out

```
-----
27-Mar-2024 09:37:06.564 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Command line argument: -Dcatalina.base=/Applications/apache-tomcat-11.0.0-M18
27-Mar-2024 09:37:06.564 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Command line argument: -Dcatalina.home=/Applications/apache-tomcat-11.0.0-M18
27-Mar-2024 09:37:06.564 INFO [main] org.apache.catalina.startup.VersionLoggerListener.log Command line argument: -Djava.io.tmpdir=/Applications/apache-tomcat-11.0.0-M18/temp
27-Mar-2024 09:37:06.582 INFO [main] org.apache.catalina.core.AprLifecycleListener.lifecycleEvent The Apache Tomcat Native library which allows using OpenSSL was not found on the java.library.path: [/Users/ornellairrera/Library/Java/Extensions:/Library/Java/Extensions:/Network/Library/Java/Extensions:/System/Library/Java/Extensions:/usr/lib/java:.]
27-Mar-2024 09:37:07.072 INFO [main] org.apache.coyote.AbstractProtocol.init Initializing ProtocolHandler ["http-nio-8080"]
27-Mar-2024 09:37:07.124 SEVERE [main] org.apache.catalina.util.LifecycleBase.handleSubClassException Failed to initialize component [Connector["http-nio-8080"]]
    org.apache.catalina.LifecycleException: Protocol handler initialization failed
        at org.apache.catalina.connector.Connector.initInternal(Connector.java:1034)
        at org.apache.catalina.util.LifecycleBase.init(LifecycleBase.java:127)
        at org.apache.catalina.core.StandardService.initInternal(StandardService.java:554)
        at org.apache.catalina.util.LifecycleBase.init(LifecycleBase.java:127)
        at org.apache.catalina.core.StandardServer.initInternal(StandardServer.java:1013)
        at org.apache.catalina.util.LifecycleBase.init(LifecycleBase.java:127)
        at org.apache.catalina.startup.Catalina.load(Catalina.java:706)
        at org.apache.catalina.startup.Catalina.load(Catalina.java:729)
        at java.base/jdk.internal.reflect.DirectMethodHandleAccessor.invoke(DirectMethodHandleAccessor.java:103)
        at java.base/java.lang.reflect.Method.invoke(Method.java:580)
        at org.apache.catalina.startup.Bootstrap.load(Bootstrap.java:299)
        at org.apache.catalina.startup.Bootstrap.main(Bootstrap.java:469)
    Caused by: java.net.BindException: Address already in use
        at java.base/sun.nio.ch.Net.bind0(Native Method)
        at java.base/sun.nio.ch.Net.bind(Net.java:565)
        at java.base/sun.nio.ch.ServerSocketChannelImpl.netBind(ServerSocketChannelImpl.java:344)
        at java.base/sun.nio.ch.ServerSocketChannelImpl.bind(ServerSocketChannelImpl.java:301)
        at org.apache.tomcat.util.net.NioEndpoint.initServerSocket(NioEndpoint.java:240)
        at org.apache.tomcat.util.net.NioEndpoint.bind(NioEndpoint.java:195)
        at org.apache.tomcat.util.net.AbstractEndpoint.bindWithCleanup(AbstractEndpoint.java:1430)
        at org.apache.tomcat.util.net.AbstractEndpoint.init(AbstractEndpoint.java:1443)
        at org.apache.coyote.AbstractProtocol.init(AbstractProtocol.java:624)
        at org.apache.coyote.http11.AbstractHttp11Protocol.init(AbstractHttp11Protocol.java:78)
        at org.apache.catalina.connector.Connector.initInternal(Connector.java:1032)
        ... 11 more
27-Mar-2024 09:37:07.125 INFO [main] org.apache.catalina.startup.Catalina.load Server initialization in [943] milliseconds
27-Mar-2024 09:37:07.185 INFO [main] org.apache.catalina.core.StandardService.startInternal Starting service [Catalina]
27-Mar-2024 09:37:07.186 INFO [main] org.apache.catalina.core.StandardEngine.startInternal Starting Servlet engine: [Apache Tomcat/11.0.0-M18]
27-Mar-2024 09:37:07.198 INFO [main] org.apache.catalina.startup.HostConfig.deployDirectory Deploying web application directory [/Applications/apache-tomcat-11.0.0-M18/webapps/docs]
27-Mar-2024 09:37:07.610 INFO [main] org.apache.catalina.startup.HostConfig.deployDirectory Deployment of web application directory [/Applications/apache-tomcat-11.0.0-M18/webapps/docs] has finished in [411] ms
```

# How can we debug a web application?

---

We need to create custom logs, so that we can decide:

- how to structure our messages
- How and where these logs are saved
- Decide to keep only the logs of a specific priority

We should use **Log4j** it is a widely-used logging framework for Java applications.

**log4j** requires:

- the **log4j.xml** where a set of rules to save logs are defined.
- The **LogContext** file: this file allows to associate additional context information with logs.

# The output of log4j

---

```
2024-05-30 09:25:06,593 INFO [http-nio-8081-exec-348]
LoadingImageDAO.doAccess(LoadImageDAO.java:71)
  IP = 0:0:0:0:0:0:0:1;
    USER = ; ACTION = LOAD_FILM_PHOTO; RESOURCE =
  Photo for movie {} successfully loaded.

2024-05-30 09:25:06,595 INFO [http-nio-8081-exec-348]
LoadImageServlet.doGet(LoadImageServlet.java:71)
  IP = 0:0:0:0:0:0:0:1;
    USER = ; ACTION = LOAD_FILM_PHOTO; RESOURCE =
  Photo for movie %d successfully sent.

2024-05-30 09:25:06,598 INFO [http-nio-8081-exec-347]
LoadingImageDAO.doAccess(LoadImageDAO.java:71)
  IP = 0:0:0:0:0:0:0:1;
    USER = ; ACTION = LOAD_FILM_PHOTO; RESOURCE =
  Photo for movie {} successfully loaded.

2024-05-30 09:25:06,601 INFO [http-nio-8081-exec-347]
LoadImageServlet.doGet(LoadImageServlet.java:71)
  IP = 0:0:0:0:0:0:0:1;
    USER = ; ACTION = LOAD_FILM_PHOTO; RESOURCE =
  Photo for movie %d successfully sent.
```

This output is structured according to our configuration with predefined custom messages we places in correspondence of points where an error was more probable. This was not possible with tomcat default log implementation.

# The Presentation Layer

---

# The Presentation Layer

---

The presentation layer includes of three components.



These components form the final web page visible to the user he can interact with

# HTML

---

HTML (HyperText Markup Language) is the standard markup language for creating and structuring content on the web. It uses various elements, represented by **tags**, to define the structure and layout of a web page, instructing the browser on how to display the content.

By default, the html web pages have no style associated.



# HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <meta name="description" content="Example Bootstrap WA 2017-2018">
  <meta name="author" content="Maria Maistro">

  <title>Unusual Animals</title>
  <link rel="stylesheet" href="https://stackpath.bootstrapcdn.com/bootstrap/4.3.1/css/bootstrap.min.css" integrity=
  <link rel="stylesheet" href="https://use.fontawesome.com/releases/v5.8.1/css/all.css" integrity="sha384-50oBUHEmv
  <link href="css/style.css" rel="stylesheet">
</head>
<body>

<div class="container containerOver">

  <!-- The header is organized with a row and 4 columns:
  + Column 1 with size 1 (kiwi icon)
  + Column 2 with size 1 (empty)
  + Column 3 with size 8 (page header)
  + Column 4 with size 2 (login form)
  -->
  <header>
    <div class="row">
      <!-- Note: to the same div we assigned two different classes, col-md-1 and kiwi_icon -->
      <div class="col-md-1 kiwi_icon">
        <i class="fas fa-kiwi-bird fa-5x"></i>
      </div>
```

Html is the root element

<head> tag identifies the head of the document

<meta> provide info about a document (these info are machine readable and are used by browser and search engines)

<link> identifies a relationship between the current document and an external source

<body> contains the actual content

# CSS

---

It is the W3C standard for defining the presentation of documents written in HTML.

CSS allows you to **style** HTML elements adding a set of rules that determine how that content will appear (i.e., colors, margins, padding..)

CSS allows you to define a set of rules to make your web application **responsive**, hence able to provide a good user experience on a wide range of devices (i.e., mobile phones, computers, tablets)

There are several front-end framework such as **Bootstrap** that provide ready-to-use styled components which are also response, making the web development simpler and faster.

# CSS

---

There are three ways to incorporate style in an HTML page:

- **External style sheets:** it is created a style sheet file and it is imported in the `<head>` tag of the html web page.
- **Embedded style sheets:** style is defined in the `<head>` tag inside the tag `<style>`
- **inline style sheets:** defined directly to an html element.

If the same style is applied to internal, embedded and external style sheets, the internal has the highest priority, the embedded follows, and finally the external.

# JavaScript

---

JavaScript concerns the **behavior** of a web application.

JavaScript is a high-level, dynamically typed, interpreted programming language that is well-suited to object oriented and functional programming styles. It runs on the user's machine and not on the server.

JS allows you to define functions to:

- **Access** and **modify** the DOM in order to generate new content dynamically
- **Handle events** and define how components can react to specific events
- **Handle asynchronous requests**

# JavaScript

---

Js code can be integrated in an html element in three ways:

- Adding the `<script>` tag in the `<head>` element;
- Adding the `<script>` tag before the `</body>`
- Adding external .js files inside the head tag

# AJAX

---

## What is ajax?

- Ajax stands for **A**synchronous javascript **a**nd **x**ml and allows you to handle asynchronous http requests between client and server.
- What does it means **asynchronous**?

This means that client and server communicate behind the scenes and the user **has not to wait the entire page to be reloaded** and can keep interacting with it.

**AJAX allows single portions of the page to be updated** without the need to reload the entire page.

# AJAX

---

It employs the **XMLHttpRequest** object to establish communication with servers.

This mechanism allows the transmission and reception of data in JSON, XML, HTML, and text files.

AJAX can be used with REST resources: JSON representations are returned in response to the requests and can be used to update the current web page generating and updating the DOM.

# JavaScript and Ajax example

```
function getStudentsList(){
    const url = "http://localhost:8081/ratings/rest/reviews";
    console.log("Request URL: %s.", url)
    const xhr = new XMLHttpRequest();

    if (!xhr) {
        console.log("Cannot create an XMLHttpRequest instance.")

        alert("Giving up :( Cannot create an XMLHttpRequest instance");
        return false;
    }

    // set up the call back for handling the request
    xhr.onreadystatechange = function () {
        processReviews(this);
    };

    // perform the request
    console.log("Performing the HTTP reviews GET request.");

    xhr.open("GET", url, true);
    xhr.send();

    console.log("HTTP GET reviews request sent.");
}
```

Instantiate the XMLHttpRequest()

This is our callback function. Everytime the state changes we will try to enter there, But only when the state is == XMLHttpRequest.DONE we will be able to generate the dynamic content



# Combining JSP, HTML, CSS, JS

---

Our starting point can often be a jsp file. This file can be used by another servlet to forward the request.

In this jsp we represent the request attributes, the session information, java beans and so on.

We can incorporate css and js scripts so that it is possible to style the jsp and add the behaviour: we can associate event handlers and generate dynamic content.

In this way, can both rely on the exchange of information between servlet and jsp, but also we can represent json representations of one or more resources, importing ajax functions and asynchronous requests in the js file which is automatically imported.

# Can I debug the frontend? – Console Logs

In console you can see log messages or errors from js functions

In sources you can debug js

In networks you can see the details of all the requests

Here you have access to DOM

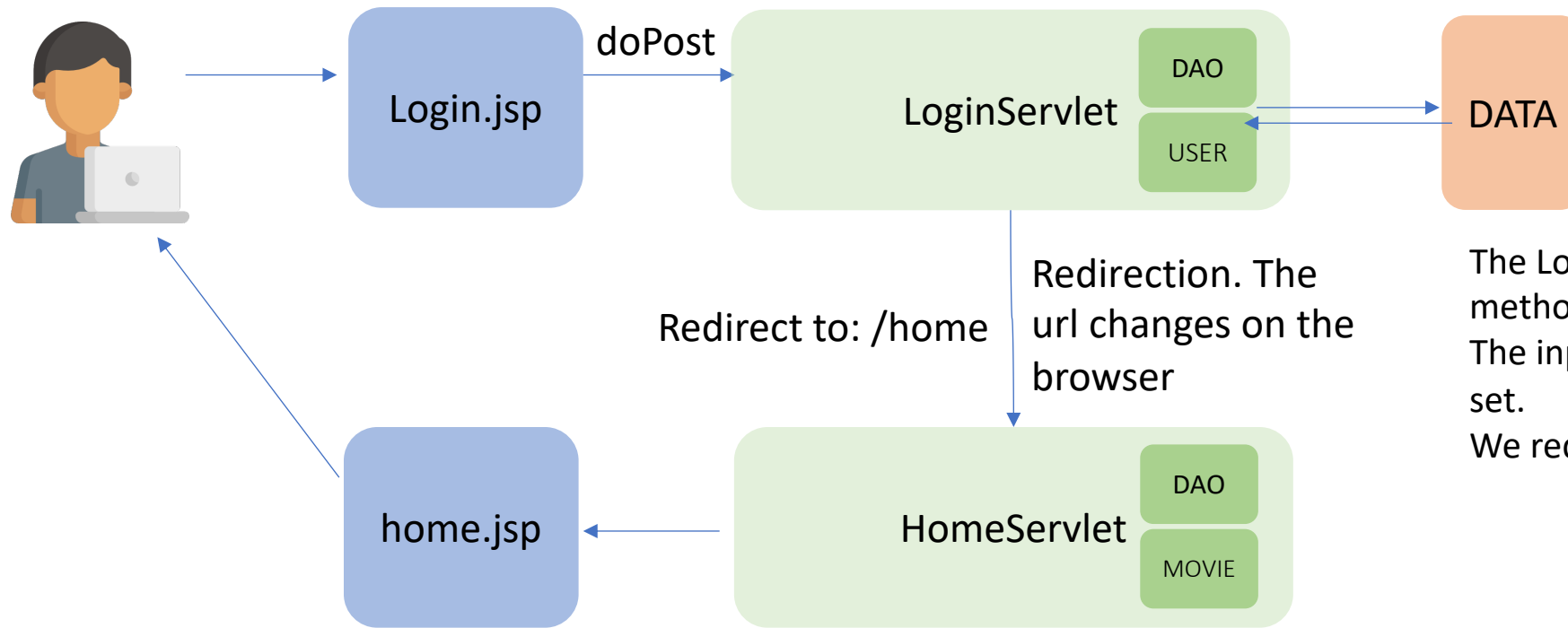
Here you have access to the style of the web page

body.skin-vector-legacy.mediawiki.ltr.sitedir-ltr.mw-hide-empty-elt.ns-

margin  
border  
padding  
406x813

# MovieRatings – login

The user interacts with the login.jsp and sends his credentials



The LoginServlet is called (doPost() method).  
The inputs are validated and the session is set.  
We redirect the user to its HOMEPAGE

Forward! The url does not change